THE UNIVERSITY OF WESTERN AUSTRALIA



CITS5206 Information Technology Capstone Project

Final Group Report

Autonomous Mining Equipment Market Map

Prepared by Group 5

Hengan Weng (24201874)
Shee Wang (24368932)
Zerun Zhang (24389925)
Hans Lionar (24661999)
Zunhao Zhang (24758673)
Tyrone Tran (24814799)

Client

Dr Adrian Boeing
UWA EECE

Facilitator

Fudong Qin

Representative users

RCT Global · Epiroc

15 October 2026

Semester 2, 2026

Contents

Resources **i**

1 Requirements and solution **1**

2 Verification and testing **2**

3 User guides and deployment **2**

4 Discussion and future work **3**

5 Use of generative AI **4**

6 References **5**

Resources

The client and the facilitator have access to every resource listed below. Issue and pull request numbers in this report, such as #212, refer to the repository: https://github.com/zinken7/capstone-group-5/issues and https://github.com/zinken7/capstone-group-5/pulls.

Resource	Link
Staging site	capstone-group-5.vercel.app
GitHub repository	github.com/zinken7/capstone-group-5
Project board	github.com/zinken7/capstone-group-5/projects
Sprint milestones	github.com/zinken7/capstone-group-5/milestones
Meeting minutes	github.com/zinken7/capstone-group-5/tree/main/docs/meeting_minutes
Project plan	github.com/zinken7/capstone-group-5/blob/main/docs/project_plan.md
Requirements (SRS)	github.com/zinken7/capstone-group-5/blob/main/docs/SRS/SRS.md
User guide	github.com/zinken7/capstone-group-5/blob/main/docs/user-guide.md
Administrator guide	github.com/zinken7/capstone-group-5/blob/main/docs/admin-guide.md
Correspondence log	github.com/zinken7/capstone-group-5/blob/main/docs/correspondence-log.md
MS Teams, Group 5 channel	Group 5

1 Requirements and solution

We built an open, crowdsourced and queryable map of autonomous and teleremote mining equipment. All 15 must-have items in the MVP scope sent to the client on 8 October are on the staging site. Twelve were working on 8 October, and the other three were closed by 15 October: operational deployments shown by default (#172, PR #180), the required submission fields (#171, PRs #188 and #193), and the four automation levels in the client's wording (#157, PR #186).

The need. Deployment data is sold in market reports at about $50,000–100,000 each, and each report is a single source that nobody can check (client meeting, 6 August [5]). No open dataset exists, so startups, analysts and researchers are locked out.

Requirements. The written brief names four functions: an interactive map, an analytics dashboard, a queryable register, and verified crowdsourcing. We turned three client meetings and two industry users' input into a specification, now at version 0.7 [1], following our project plan [3]. Every requirement records its source, and anything the team proposed without a source is marked as such. The client confirmed the main items at the third client meeting on 18 September; the written scope agreement sent on 8 October has had no reply, and a revision matching the build is put to him for signature at the final demonstration on 16 October [2].

Solution. The system is written in TypeScript: a React front end with a Leaflet map, and a Node API using Drizzle ORM over PostgreSQL with PostGIS. Staging runs on Vercel and redeploys from the main branch. Public pages show verified records only; a record needs at least one evidence link and site coordinates before it can be verified, and nothing is published until an administrator approves it.

Function	Delivered on staging (15 Oct)
Map	One marker per site with detail (FR-15, FR-17); operational deployments by default (FR-16); supplier filter (FR-18); year control (FR-19); new sites appear after approval (FR-04)
Analytics	Headline counts, machines in service over time, counts by country and machine type, market share (FR-21–26); one filter set for every chart, aggregates only (FR-27, FR-28)
Equipment register	Search, filter and sort (FR-36); export to CSV (FR-37)
Contribute and review	Public form with no account (FR-40) and the required fields (FR-07, FR-08); nothing public until approved (FR-42); four automation levels (FR-57); evidence links (FR-44); approve, reject, request changes and correct, with a duplicate warning for reviewers (FR-45, FR-46, PR #192); submission history for administrators (PR #191)
All pages	Accessibility to WCAG 2.1 AA (NFR-01)
Optional: evidence check (FR-55)	Separate module that checks a submission against its evidence link; connection to the review screen awaits the client's decision

Beyond that scope, staging closes the review loop: contributors follow their submissions and answer a request for changes with a private key, without an account, and resubmit a rejected record as a linked one (FR-59 to FR-61, PRs #215 to #220 and #224). It also has extra map filters (country, machine type, automation level) and a breakdown of machines by automation level. The *Should* requirements not delivered this semester are listed in Section 7 of the specification [1].

2 Verification and testing

Every change passed automated tests in CI and a review by another member before it reached main, and then ran on staging before the client saw it.

Activity	What it checks	Evidence
Automated tests in CI	Every pull request runs the backend and frontend suites, and none merges unless they pass. On main at submission on 15 October, the backend has 421 tests and the frontend 283; `npm test` in `backend/` or `frontend/` runs them.	ci.md
Schema check in CI	The build fails if the code and the database schema drift apart. Moving the data layer to Drizzle ORM caught two bugs.	PR #102
Code review	No pull request merges without another member's approval, and reviewers reproduce a problem before reporting it: a reported data leak was reproduced locally and fixed before merge.	PR #173
Manual QA	Each page checked on staging against a written checklist, by a member other than its author.	QA checklist
Accessibility	Automated end-to-end accessibility tests, plus manual checks of contrast, colour-independent meaning, keyboard use and labelled forms, to WCAG 2.1 AA.	accessibility.spec.ts, PR #169
Workflow and security testing	The whole submit-and-review workflow was run end to end on staging. It found that anyone could edit or reopen an unverified record without signing in. The route now refuses such edits (403), confirmed on staging before and after the fix.	#211, PR #212
Client acceptance testing	The client reviewed the build at three client meetings and received the MVP scope agreement with its acceptance checklist on 8 October. The revised agreement, matching the build, is put to him for signature at the final demonstration on 16 October; his answers to its open questions are recorded in the correspondence log.	Correspondence log, meeting minutes

Evidence check (FR-55). The separate engine, designed in [4] and built in PR #85, has 225 automated tests. On 20 labelled cases containing 110 facts, its free local check judged 105 correctly, and all 110 when Claude Haiku settled the uncertain ones. Harder cases written by a second member found three weaknesses, listed in Section 4.

3 User guides and deployment

To run the system, follow the README. It was tested by following it step by step on a clean Ubuntu machine, macOS and Windows (#189, PR #190), and it shows how to create an administrator locally, so every part of the system can be tried without our credentials.

Document	For	What it covers	Link
README	Developers, markers	Requirements, getting the code, starting the database, backend and frontend, creating an administrator, environment variables, troubleshooting	README
User guide	Public users and contributors	The map, register and analytics; submitting a record with evidence; following a submission with its private key and answering a request for changes	user-guide.md
Administrator guide	Reviewers	Signing in; the review queue; approving, rejecting, requesting changes and correcting a record	admin-guide.md

Document	For	What it covers	Link
Deployment and CI	Whoever hosts it next	How staging deploys from main, database migrations, CI checks	staging.md, ci.md
Developer references	Developers	API endpoints, data model and architecture, branch and commit conventions	API, ERD, architecture, CONTRIBUTING.md

Repository layout: `backend/` holds the API and its tests, `frontend/` the web app with its unit and end-to-end tests, and `docs/` the specification, guides and meeting records.

Staging: capstone-group-5.vercel.app, with placeholder data, not the client's records. Administrator credentials are given to the client separately and are not in this report.

Licences: MIT for the source code and CC BY 4.0 for the dataset, as proposed to the client; his confirmation is sought on 16 October.

4 Discussion and future work

All 15 planned must-haves are delivered, and staging goes beyond them with the closed review loop (Section 1). Each remaining gap is recorded as an open issue in the repository (issue list).

Open issues at submission:

- #83 and #69: the evidence check works as a separate module (PRs #84 and #85) but is not connected to the review screen. Connecting it needs the client's decision, a rebase onto main, a guard on outbound requests, and a background worker, which the current serverless host cannot run.
- Rate limiting **[#244]**: a hidden honeypot field stops simple bots (FR-51, PR #165), but the public forms have no rate limiting, so a script could still flood the review queue.
- *Should* requirements not delivered **[#245]**: FR-12 in part, FR-13, FR-32, FR-33, FR-38, FR-49 and FR-50, each recorded in Section 7 of the specification [1].
- Staging deployment **[#246]**: if a deploy fails after its database migration has run, staging keeps the new schema with the previous code until the job is re-run or the next merge deploys.

What worked. Tracing every requirement to its source kept scope discussions short: when a feature was questioned, the source settled it. Delivering one complete slice per sprint meant there was always something to show the client.

What we would change. We tested the review workflow end to end late. Done earlier, it would have found the anonymous-edit flaw sooner: a unit test had asserted that behaviour as correct, because the specification did not yet say who may change a record before it is verified. Staging redeploys only from merges made by one member's account, which made that member a bottleneck for every release.

Future work:

1. Connect the evidence check to the review screen, if the client agrees; fix its three known weaknesses (planned deployments read as current, fleets credited to the wrong site, confused company relationships), then extend it with source dates, page language and reviewers' decisions.
2. Add rate limiting to the public forms.
3. Deliver the remaining *Should* items, starting with FR-13, FR-32 and FR-33, which the data model already supports.
4. Replace the placeholder data with real records from industry contributors.
5. Move hosting to a UWA environment and confirm the licences.

5 Use of generative AI

We used generative AI in documents, code and review, under two team rules.

- **No client data in AI tools.** The client's dataset was never given to an AI tool, and API keys stayed in local environment files.
- **A person reviews every merge.** AI output goes through the same review and tests as any other change.

AI inside the product. The evidence check uses a small local language model, and calls Claude Haiku only when the local model is unsure. The model may only point to a sentence number in the source, never write a quote, and the code then checks that the sentence exists. The engine recommends; it never changes a record's status, and an administrator makes every decision.

Member	Where AI was used	How the output was checked and changed
Hans Lionar	Claude, as drafting partner for the project-management work: the specification, the scope agreement, this report, the presentation, pull-request reviews, and team messages, which I dictate as intent and have rendered in plain English so that teammates reading in a second language get the same meaning. Claude Code, for the evidence-check engine and the Drizzle migration to my design, and for a first pass over teammates' pull requests before I read them. Agent-written commits carry a co-author line.	I decide and the AI drafts. Checking follows priority, not habit: must-have features and security findings are reproduced before I act, documents are read before they go out, and low-stakes drafts such as routine team messages go out on a skim. Reproduced first: a reported data leak confirmed locally and fixed (PR #173); a suggested map bug dropped when it could not be reproduced (PR #179); the anonymous-edit hole reproduced on staging before the fix and confirmed refused after it (#211, PR #212). The evidence-check engine, a Could item, I checked by behaviour only; its results in Section 2 rest on its test suite, the 20 cases I labelled and five harder ones written by a teammate.
Tyrone Tran	OpenAI Codex to generate the early prototype and suggest optimisations; Claude Code to gather repository and meeting evidence and translate Vietnamese notes into English for the report.	I defined the prototype's scope and data model, then ran, debugged and checked it before the client demonstration. I assessed optimisation suggestions against project requirements and architecture, verified collected evidence and revised translated text. For example, AI suggested a button to return rejected submissions to the review queue. I rejected this approach because it would create unnecessary manual work for administrators. I instead designed a workflow covering approval, rejection and requests for changes (PR #210), with contributor updates returning submissions to review (PR #216) and rejected submissions resubmitted as new linked records (PR #218).
Zunhao Zhang	I used ChatGPT to generate 200 simulated seed records based on the updated database structure and generation rules for software development and testing. No client data was uploaded; only the schema and generation rules were provided. The generated records were used as test data, not as real deployment data for the live system.	I checked the generated records against the field requirements and relationship rules in the updated ERD and verified that they could be imported into the database. For example, I found that some simulated records used `line_of_sight_remote`, which was no longer an accepted automation-level value in the updated model. I changed these values to `teleremote` and updated the related technology products to keep the test data consistent.

Member	Where AI was used	How the output was checked and changed
Shee Wang	Codex/ChatGPT, for checking frontend code and improving English wording. I used it to review UI changes in the map, Equipment register and contribution/correction forms, and to organise explanations for the report and presentation.	I checked suggestions against the code, tests, staging behaviour and project requirements. I made the final decisions and edits myself. For example, I rejected a suggested mobile map layout change when it did not preserve the existing marker focus and detail-panel behaviour, and I rewrote AI-provided English before adding it to the report.
Zerun Zhang	Claude Code, to help probe the backend for concurrency, injection and data-exposure problems. For concurrency, it drafted and ran a test that forces two corrections onto the same record at once. For injection, it checked that queries are parameterised and that the sort allow-list, search wildcard escaping and CSV formula protection work, with tests covering each. For vulnerabilities, it reproduced an unreviewed submission moving a site on the public map (PR #173) and a stale admin session after reactivation (PR #105).	For the concurrency test, the assistant checked that it fails when the row lock is removed, and I reviewed the result. I reviewed both reproductions before reporting them on those pull requests. I chose which features and pull requests to test, read every finding before acting on it, and only raised issues whose reproduction I had checked. Changed: every fix was undone once to confirm its tests fail, and any test that still passed was rewritten; in PR #166 a drafted test assumed a site had a commissioning year, and I had its expectation corrected after it failed. Rejected: the assistant once merged main into my branch after I had declined it; I had the merge removed before anything was pushed and set a rule against it.
Hengan Weng	Codex/ChatGPT, for technical guidance, debugging, testing, code review, and documentation across various project tasks.	I independently reviewed and verified AI suggestions through code inspection, testing, and CI. For Issue #168, I checked the suggested implementation against the existing code and reviewer feedback, revised the changes rather than accepting the initial output unchanged, and validated the updated implementation through testing and CI before merging PR #192.

6 References

[1] Group 5, "Software Requirements Specification," v0.7, CITS5206, Univ. of Western Australia, Oct. 15, 2026. [Online]. Available: https://github.com/zinken7/capstone-group-5/blob/main/docs/SRS/SRS.md

[2] Group 5, "MVP Scope Agreement," sent to the client Oct. 8, 2026, revised for signature Oct. 16, 2026. [Online]. Available: https://github.com/zinken7/capstone-group-5/blob/main/docs/correspondence-log.md

[3] Group 5, "Project Plan and Specification Report," CITS5206 Assignment 1, Univ. of Western Australia, Aug. 25, 2026. [Online]. Available: https://github.com/zinken7/capstone-group-5/blob/main/docs/d1/submitted/2026-08-25-group5-assignment-1.pdf

[4] Group 5, "FR-55 Automated Evidence Check: Design," pull request #84, Oct. 2026. [Online]. Available: https://github.com/zinken7/capstone-group-5/pull/84

[5] Group 5, "Client and facilitator meeting minutes," Aug. 6 to Sep. 29, 2026. [Online]. Available: https://github.com/zinken7/capstone-group-5/tree/main/docs/meeting_minutes